

Sparse Matrix-Vector Multiplication on GPU: A Format and Kernel Strategy Comparison

Luca Agostinelli

Abstract—An experimental study of Sparse Matrix-Vector Multiplication (SpMV) on an NVIDIA A30 GPU (Ampere architecture, peak memory bandwidth 933 GB/s) is presented. Four main GPU kernels are evaluated across 10 SuiteSparse matrices.

The evaluated kernels include a CSR Scalar/Vector Adaptive kernel combining scalar and warp-reduction paths, the Hybrid ELL+COO format (HYB) from Bell & Garland [1], and the NNZ-splitting approach with auto-selection (ACC) from Chu et al. [2]. NVIDIA cuSPARSE is used as a vendor reference.

Results show that no single kernel dominates across all matrix types. On structurally regular matrices, HYB and ELL achieve the highest throughput; NNZ-splitting methods are more robust on irregular, power-law-distributed graphs. Memory-access coalescing and row-length imbalance are identified as the dominant performance factors across all kernels.

I. INTRODUCTION

Sparse Matrix-Vector Multiplication (SpMV) computes $y = Ax$, where A is a sparse matrix of $m \times n$ and x, y are dense vectors. It is one of the most pervasive kernels in scientific computing, appearing as the dominant cost in applications ranging from iterative linear solvers (e.g., Krylov methods) to graph analytics (e.g., PageRank) [3].

Despite its simplicity, SpMV is limited by *memory bandwidth* rather than compute. Each nonzero requires reading a value, a column index, and an indirect access to x , resulting in very low arithmetic intensity relative to the Roofline model. On GPUs, performance further depends on coalesced memory accesses, which are hard to achieve due to irregular sparsity patterns [1] [3].

This work presents an experimental study of SpMV on an NVIDIA A30 GPU. I implemented and evaluated different GPU kernels spanning multiple designs: (i) a *CSR Scalar/Vector Adaptive* kernel that routes short rows to a scalar path and long rows to a warp-reduction path; (ii) the *HYB* (ELL+COO) hybrid format from Bell & Garland [1], which trade padding overhead for coalesced access patterns; (iii) the *Flat* and *Line-enhance* methods from Chu et al. [2], combined in the auto-selecting *ACC* dispatcher; (iv) NVIDIA's *cuSPARSE* library as a vendor reference.

The central question of this work is how sparse storage format and parallelisation strategy affect SpMV performance on modern GPUs, and which matrix characteristics determine the best approach. In particular, I study whether load imbalance or non-coalesced memory accesses is the dominant bottleneck, and whether format-based methods or NNZ-splitting approaches are more robust across different types of matrices.

II. METHODOLOGY

A. Hardware Setup

All experiments run on a single NVIDIA A30 GPU (Ampere, compute capability 8.0) with a theoretical peak memory bandwidth of 933 GB/s. Kernels are compiled with `nvcc`, CUDA 11.8, targeting `sm_80`; host code uses C++17.

B. Input Representation and CPU Baseline

All matrices are read from Matrix Market (.mtx) format. Symmetric matrices are stored explicitly (both triangular halves), so all kernels operate on the full pattern. The dense vector $x \in \mathbb{R}^n$ is generated once with a fixed random seed (MT19937, uniform in $[-1, 1]$) and reused across all kernels. All arithmetic uses 32-bit floating point (Float32).

A sequential CSR-based SpMV is used exclusively for correctness validation. The GPU result is accepted if $|y_i^{\text{GPU}} - y_i^{\text{CPU}}| \leq \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \cdot \max(|y_i^{\text{CPU}}|, |y_i^{\text{GPU}}|)$ for all i , with $\varepsilon_{\text{abs}} = \varepsilon_{\text{rel}} = 10^{-2}$. For a few matrix entries, the tolerance is exceeded due to Float32 accumulation errors, not algorithmic faults.

C. GPU Kernels

1) **CSR Scalar/Vector Adaptive**: Based on the scalar and vector CSR kernels of Bell & Garland [1]. A one-time CPU preprocessing pass ($\mathcal{O}(m)$) partitions rows into two disjoint sets based on threshold $T=32$ (warp size). Short rows ($\text{nnz}_i < T$) are handled by the *scalar* path (Algorithm 1): one thread per row iterates sequentially over its nonzeros, writing directly to $y[\text{row}]$ with no atomics. Long rows ($\text{nnz}_i \geq T$) use the *vector CSR kernel* (one warp per row) with `__shfl_down_sync` reduction (Algorithm 2). Both kernels write to disjoint rows of y and require no synchronisation between them.

Algorithm 1: CSR scalar kernel

```
tid ← blockIdx.x · blockDim.x + threadIdx.x;
if tid < |short_rows| then
    row ← short_rows[tid];
    sum ← 0;
    for k ← row_ptr[row] to row_ptr[row + 1] - 1 do
        sum += val[k] · x[col[k]];
    y[row] ← sum;
```

Algorithm 2: CSR vector kernel

```

warp_id ← (blockIdx.x · blockDim.x + threadIdx.x) / 32;
lane ← threadIdx.x mod 32;
if warp_id < |long_rows| then
  row ← long_rows[warp_id];
  sum ← 0;
  for k ← row_ptr[row] + lane to
    row_ptr[row + 1] - 1, step 32 do
    | sum += val[k] · x[col[k]];
  for offset ∈ {16, 8, 4, 2, 1} do
    | sum +=
    | __shfl_down_sync(sum, offset);
  if lane = 0 then
    | y[row] ← sum;

```

2) **HYB (Hybrid ELL+COO)**: Bell & Garland [1] also proposed the HYB format. The matrix is split into an ELL part of width K (column-major, $K \times m$, coalesced access) and a COO tail for the overflow nonzeros of rows with more than K entries, processed with `atomicAdd` (Algorithm 3). The width K is chosen with the Bell & Garland greedy criterion: add a K -th ELL column while the number of rows with at least K nonzeros exceeds $\max(4096, m/3)$. This threshold is derived from the empirical assumption in that work that a fully-occupied ELL format is roughly $3\times$ faster than COO.

Algorithm 3: HYB SpMV

```

K ← CHOOSEELLWIDTH(m, row_ptr);
// ELL pass (one thread per row,
// column-major layout)
row ← blockIdx.x · blockDim.x + threadIdx.x;
if row < m then
  sum ← 0;
  for c ← 0 to K - 1 do
    | idx ← c · m + row;
    | sum += ell_val[idx] · x[ell_col[idx]];
  y[row] ← sum;
// COO tail pass (one thread per
// overflow NNZ)
tid ← blockIdx.x · blockDim.x + threadIdx.x;
if tid < coo_nnz then
  | atomicAdd(&y[coo_row[tid]], coo_val[tid] ·
  | x[coo_col[tid]]);

```

3) **Flat, Line-enhance, and ACC on CSR**: These kernels implement the NNZ-splitting approach of Chu et al. [2]; I refer to that paper for full algorithmic details (Algorithms 2-5). My implementations are direct translations of the published pseudocode, with one correction to Algorithm 3 (line 16): `row_ptr[reduce_row_end] mod N = 0` must be replaced by `row_ptr[reduce_row_end] mod N ≠ 0`.

Flat ($R=4$, $\text{THREADS}=256$) partitions the CSR nonzeros into fixed-size NNZ chunks, uses a GPU preprocessing kernel to build a break-point array, and accumulates partial row sums with `atomicAdd`. **Line-enhance** ($R=2$, $\text{THREADS}=512$) assigns complete rows to each block and reduces with `VecReduce` ($V \in \{1, 2, 4\}$ chosen by $\bar{d}$), writing directly to y with no atomics. **ACC** is an adaptive dispatcher that extends the strategy of Chu et al. [2]: while the original routes between Flat and Line-enhance based solely on matrix size and average row density $\bar{d}$, my implementation additionally incorporates a row-length imbalance criterion, as shown in Algorithm 4.

4) **cuSPARSE (reference)**: NVIDIA’s cuSPARSE library (ALG_DEFAULT on a CSR descriptor) serves as the vendor-tuned performance ceiling.

Algorithm 4: ACC heuristic dispatcher

```

Compute average row density  $\bar{d} \leftarrow \text{nnz}/m$ ;
Compute imbalance ratio  $\rho \leftarrow d_{\max}/\bar{d}$ , where  $d_{\max}$  is
the maximum row NNZ;
if nnz >  $10^6$  and ( $\rho > 32$  or  $\bar{d} > 8$ ) then
  | dispatch FLAT;
else if  $\bar{d} > 128$  then
  | dispatch FLAT;
else
  | dispatch LINE-ENHANCE;

```

III. DATASET

Experiments use 10 matrices from the SuiteSparse Matrix Collection, selected to span a wide range of structural properties. Table I summarises the key statistics.

TABLE I: Dataset: SuiteSparse matrices. $\bar{d}$ = avg. NNZ/row; $d_{\max}$ = max NNZ/row; $\rho = d_{\max}/\bar{d}$ = imbalance ratio; sym. = symmetric.

Matrix	Rows	NNZ	$\bar{d}$	$d_{\max}$	ρ	sym.
crankseg_2	63 838	14 148 858	221.6	3 423	15.4	yes
poisson3Da	13 514	352 762	26.1	110	4.2	no
cage13	445 315	7 479 343	16.8	39	2.3	no
parabolic_fem	525 825	3 674 625	7.0	7	1.0	yes
thermal2	1 228 045	8 580 313	7.0	11	1.6	yes
ASIC_680k	682 862	3 871 773	5.7	395 259	69 712	no
amazon0302	262 111	823 251	3.1	142	45.2	no
webbase-1M	1 000 005	3 105 536	3.1	4 700	1 513	no
cit-Patents	3 774 768	11 012 632	2.9	267	91.5	no
roadNet-CA	1 971 281	3 688 810	1.9	7	3.7	yes

The dataset spans two orthogonal axes. Along *regularity*, matrices range from nearly uniform (parabolic_fem, $\rho=1.0$; thermal2, $\rho=1.57$) to extremely skewed (ASIC_680k, $\rho=69\,712$; webbase-1M, $\rho=1\,513$; cit-Patents, $\rho=91$). Along *density*, $\bar{d}$ covers two orders of magnitude from ≈ 1.9 (roadNet-CA) to ≈ 222 (crankseg_2). This diversity is intentional: highly regular matrices stress format-based approaches (ELL), power-law graphs expose load imbalance and padding pathologies, and large block-dense FEM matrices (crankseg_2) favour warp-parallel execution.

TABLE II: Execution time (t=ms) and Variance comparison. Variance (var=ms²) computed as population variance over 100 timed iterations. The winning time is highlighted in bold.

Matrix	ACC		S/V-Adaptive		HYB		cuSPARSE	
	t (ms)	var	t (ms)	var	t (ms)	var	t (ms)	var
ASIC_680k	0.2	3.44e-07	1.9	4.41e-02	2.1	5.96e-05	0.08	5.42e-07
amazon0302	0.03	3.18e-07	0.04	3.44e-07	0.04	2.66e-07	0.04	2.36e-07
cage13	0.25	1.12e-06	0.15	3.19e-06	0.12	5.60e-07	0.12	7.16e-07
cit-Patents	0.72	6.85e-07	0.62	9.24e-07	0.75	1.43e-06	0.57	1.79e-06
crankseg_2	0.32	1.24e-06	0.18	2.57e-07	0.45	3.33e-05	0.20	1.69e-06
parabolic_fem	0.08	5.61e-07	0.07	1.76e-06	0.05	2.50e-07	0.08	5.86e-07
poisson3Da	0.02	1.55e-07	0.02	1.91e-07	0.02	2.20e-07	0.02	1.67e-07
roadNet-CA	0.12	2.80e-07	0.12	2.05e-07	0.12	3.48e-07	0.11	7.60e-07
thermal2	0.16	1.36e-06	0.19	5.55e-07	0.16	9.20e-07	0.18	9.50e-07
webbase-1M	0.19	1.90e-07	0.11	7.60e-07	0.31	8.70e-06	0.08	4.99e-07

IV. RESULTS AND DISCUSSION

A. Measurement Methodology

Timing is measured with `cudaEvent`, excluding one-time setup costs. Each kernel uses 20 warm-up iterations; reported values are the mean over 100 timed launches (min/max recorded). GFLOP/s uses the standard definition $\frac{2 \cdot nnz}{t_{avg} \cdot 10^9}$, where t_{avg} is measured in seconds. Effective bandwidth is computed as $BytesMoved/t_{avg}$, where $BytesMoved = 4(2 \cdot nnz + m + 1 + n + m)$ bytes for CSR-based kernels (Float32 values, Int32 column indices and row pointers, input vector x , and output vector y); for ELL/HYB the padded slot count replaces nnz for the matrix arrays, and each COO `atomicAdd` is modelled as one read plus one write on y . The vector x is conservatively counted at full length n .

Nsight Compute (`ncu`) is used to collect per-kernel memory-behaviour metrics. Profiling is performed with `--launch-skip 20` and `--launch-count 1`, thereby excluding the initial warm-up executions and capturing a representative steady-state kernel launch. The metrics collected are: L1 TEX and L2 hit-rates, global-load coalescing efficiency, warp occupancy, DRAM read/write traffic, sectors per load request, and global load/store instruction counts.

Raw benchmark and `ncu` output files are processed by two Python scripts (`parse_results.py` and `parse_ncu.py` respectively) into CSV tables used for the analysis; complete results are omitted for brevity and available at [4].

Figure 1 and Table II reports mean execution times, while Figure 2 reports the achieved GFLOP/s. Table III reports effective bandwidth. Table IV reports NCU metrics for three representative matrices (full NCU metrics are available at [4]).

TABLE III: Effective memory bw (GB/s) and percentage of peak (933 GB/s).

Matrix	ACC		S/V Adap.		HYB		cuSPARSE	
	GB/s	%pk	GB/s	%pk	GB/s	%pk	GB/s	%pk
ASIC_680k	211.9	22.7	21.6	2.3	27.9	3.0	504.2	54.0
amazon0302	283.0	30.3	288.7	30.9	345.9	37.1	245.0	26.3
cage13	266.3	28.5	440.9	47.3	655.3	70.2	538.2	57.7
cit-Patents	207.1	22.2	239.2	25.6	276.2	29.6	232.6	24.9
crankseg_2	359.0	38.5	645.8	69.2	357.6	38.3	575.7	61.7
parabolic_fem	447.2	47.9	512.4	54.9	655.2	70.2	471.9	50.6
poisson3Da	153.6	16.5	146.6	15.7	181.7	19.5	160.1	17.2
roadNet-CA	456.3	48.9	509.7	54.6	479.8	51.4	469.7	50.3
thermal2	506.7	54.3	462.4	49.6	528.6	56.7	473.6	50.8
webbase-1M	218.3	23.4	372.5	39.9	149.9	16.1	466.3	50.0

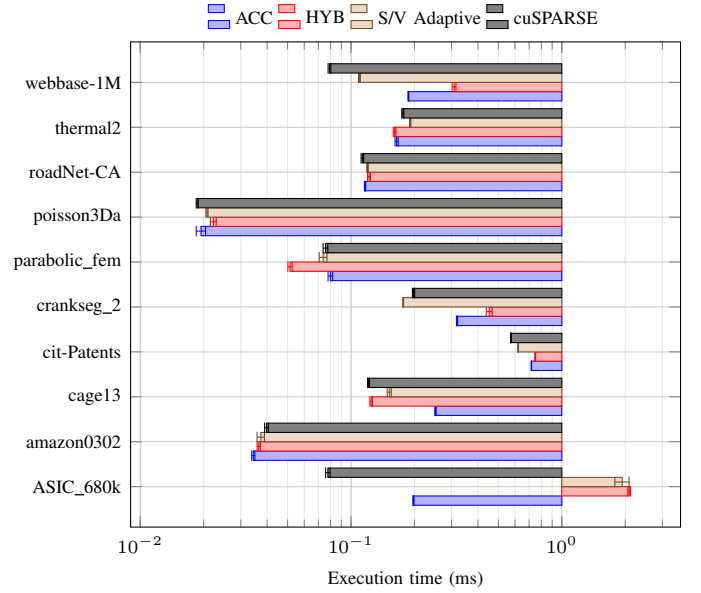


Fig. 1: SpMV execution time (mean $\pm$ min/max range) on log scale. Error bars show the observed min-max spread over 100 timed iterations.

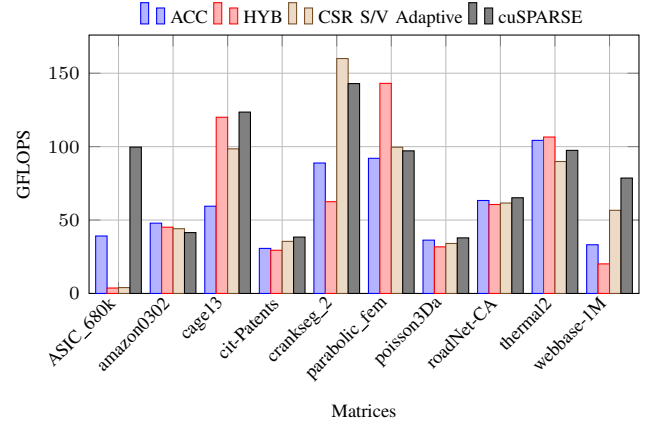


Fig. 2: Comparison of SpMV performance in GFLOPS

B. Row-length regularity determines format efficiency

Format-based kernels (ELL and HYB) perform best when row lengths are nearly uniform, because padding waste is negligible and the column-major ELL layout delivers fully coalesced 128-byte transactions per warp. On `parabolic_fem` ($\rho=1.0$) and `thermal2` ($\rho=1.6$), where every row has 7 NNZs, HYB reaches 143.1 and 106.6 GFLOP/s respectively (up to $1.47\times$ cuSPARSE on the former). The NCU data confirm the mechanism: ELL coalescing on `parabolic_fem` is 61.5% [4], and the high effective bandwidth (~ 655 GB/s) partly reflects L1/L2 cache reuse on the small x vector.

Conversely, on power-law graphs, such as `ASIC_680k` with $\rho=69712$ and `webbase-1M` with $\rho=1513$, the Bell & Garland criterion yields a very small K (3 and 2 respectively, see [4]), so 47% and 36% of NNZs overflow to the COO tail. The `atomicAdd`-based COO kernel dominates total runtime, driving HYB throughput down to 3.7 and 20.1 GFLOP/s (only 3% and 16% of peak bandwidth).

TABLE IV: Nsight Compute metrics (single kernel launch). L1/L2: hit-rate (%); coal.: global-load coalescing (%); occ.: warp occupancy (%); sec/req: sectors per load request. Full NCU metrics at [4].

Matrix	Kernel	L1%	L2%	coal.%	occ.%	sec/req
poisson3Da	Adaptive_scalar	83.26	58.72	13.36	12.29	20.76
	Adaptive_vector	41.65	64.59	31.38	40.97	6.72
	LINE	63.50	43.27	33.28	42.76	8.60
	FLAT_core	57.68	40.99	33.70	64.01	9.34
	HYB_ell	45.60	20.79	52.98	12.44	5.33
	HYB_coo	17.39	73.28	41.19	39.81	9.28
roadNet-CA	Adaptive_scalar	60.15	45.64	35.35	85.18	5.24
	LINE	44.09	47.65	55.54	94.22	5.34
	FLAT_core	37.70	39.03	57.25	87.45	5.52
	HYB_ell	34.18	38.56	59.67	94.44	4.81
	HYB_coo	8.43	33.84	48.75	85.36	7.81
	Adaptive_scalar	62.91	37.04	37.67	80.93	4.62
webbase-1M	Adaptive_vector	54.03	27.92	49.55	71.15	4.84
	LINE	47.54	37.69	66.74	79.47	4.66
	FLAT_core	31.98	31.56	67.61	84.34	4.39
	HYB_ell	13.89	32.59	76.91	88.98	4.25
	HYB_coo	11.45	65.99	63.80	87.39	6.02

C. Load imbalance is catastrophic for row-centric kernels

All kernels that assign work by row (Scalar, S/V Adaptive, HYB-ELL) are vulnerable to load imbalance: a single long row pins one warp for the entire kernel duration while the rest of the SM sits idle. On ASIC_680k ($d_{\max}=395\,259$), S/V Adaptive reaches only 4.0 GFLOP/s at 2.3% of peak bandwidth, despite routing 99.9% of rows to the fast scalar subkernel. The bottleneck is the vector subkernel: with only 424 long rows, the GPU launches 424 warps across 56 SMs (far too few to hide memory latency) and NCU confirms this with 7.5% warp occupancy [4].

NNZ-splitting (Flat) decouples work from row structure entirely, assigning a fixed number of nonzeros per block regardless of row boundaries. ACC dispatching to Flat is therefore the only custom kernel that handles ASIC_680k effectively, reaching 39.1 GFLOP/s.

Notably, ASIC_680k is the only matrix where cuSPARSE achieves a decisive lead over all custom kernels.

D. Bandwidth is mainly determined by coalescing efficiency

All kernels are memory-bound: effective bandwidth ranges from 2% of peak (ASIC_680k, S/V Adaptive) to 70% (cage13/parabolic_fem, HYB). The primary differentiator is global-load coalescing. Row-centric kernels achieve only 13-37% (sectors/request up to 20), because indirect gathers into $\mathbf{x}$ follow the graph’s random adjacency structure. NNZ-splitting and ELL layouts raise coalescing to 34-77% (sectors/request ≈ 4 -5).

On webbase-1M ($\text{avg_nnz}=3.1$), however, coalescing alone does not determine the winner: Line-enhance achieves 66.7% coalescing yet is the slowest kernel, likely because VecReduce and shared-memory coordination dominate over useful arithmetic. S/V Adaptive, routing nearly all rows through the scalar subkernel, reaches 56.6 GFLOP/s ($9\times$ Line-enhance).

E. ACC heuristic: accuracy and limitations

ACC correctly dispatches on 7 of 10 matrices. On the remaining three (cage13, cit-Patents, crankseg_2) it selects Flat over the faster Line-enhance, because the $\bar{d}$ -based threshold ignores Flat’s preprocessing overhead on moderately dense, low-imbalance matrices. The added imbalance-ratio criterion ($\rho > 32$) correctly redirects ASIC_680k and webbase-1M to Flat, avoiding the mis-dispatch the original $\bar{d}$ -only rule of Chu et al. [2] would have produced.

F. cuSPARSE as performance upper bound

cuSPARSE ranks first or within 10% of the best custom kernel on 7 of the 10 matrices and is never the worst on any, confirming its robustness as a vendor-tuned reference. Custom kernels surpass it only on structurally regular matrices: S/V-Adaptive reaches 160.0 GFLOP/s on crankseg_2 ($1.12\times$) and HYB reaches 143.1 GFLOP/s on parabolic_fem ($1.47\times$). On power-law graphs such as ASIC_680k and webbase-1M, cuSPARSE leads by $2.55\times$ and $1.39\times$ respectively, suggesting internal heuristics that the custom implementations lack.

V. CONCLUSIONS

No single kernel or format dominates SpMV across structurally diverse matrices; the optimal choice depends on row-length regularity and load balance. On FEM-like matrices with nearly uniform row lengths, ELL and HYB saturate bandwidth efficiently and can outperform cuSPARSE, as the column-major ELL layout delivers fully coalesced warp transactions with negligible padding waste. On power-law graphs, row-centric kernels degrade severely: a single dominant row serialises one warp while occupancy collapses. NNZ-splitting is the only robust strategy, assigning fixed nonzero chunks per block regardless of row boundaries; cuSPARSE still leads on the most skewed matrices, suggesting internal adaptive logic the custom kernels lack. Coalescing is the second major factor: ELL and Flat achieve sectors/request ≈ 4 -5 versus ≤ 20 for row-centric kernels. On very short rows, VecReduce synchronisation overhead dominates, making the scalar path faster. The ACC dispatcher selects correctly on 7 of 10 matrices; mis-dispatches occur where Flat preprocessing cost outweighs its load-balancing benefit.

REFERENCES

- [1] N. Bell and M. Garland, “Implementing sparse matrix-vector multiplication on throughput-oriented processors,” in *Proceedings of SC’09*, 2009.
- [2] G. Chu et al., “Efficient algorithm design of optimizing spmv on gpu,” in *Proceedings of HPDC’23*, 2023. DOI: 10.1145/3588195.3593002.
- [3] J. Gao, B. Liu, W. Ji, and H. Huang, “A systematic literature survey of sparse matrix-vector multiplication,” *arXiv preprint arXiv:2404.06047*, 2024.
- [4] L. Agostinelli, *Source code and complete results*, <https://github.com/LucaAgostinelli/GPU-d1>, 2026.